

Contents

1. Notes	2	4.11. GomoryHu	13
1.1. Chuỗi Taylor	2	4.12. MinCostMaxFlow	13
1.2. Catalan	2	4.13. GlobalMinCut	14
1.3. Lucas	2	5. Xâu	14
1.4. Bổ đề Burnside	2	5.1. KMP	14
1.5. Định lý Pick	2	5.2. Z	14
1.6. Frobenius	2	5.3. MinRotation	14
1.7. Super interpretation of kth powers	2	5.4. SuffixArray	15
1.8. Nhận xét	2	6. Quy hoạch động	15
2. Toán	2	6.1. CountSubseq	15
2.1. ModInt	2	6.2. SOSDP	15
2.2. MillerRabin	3	6.3. 1D1D	16
2.3. Matrix	3	6.4. Knuth	16
2.4. ModLog	3	6.5. DnCDP	17
2.5. ModSQRT	3	7. Khác	17
2.6. Factor	3	7.1. maxHist	17
2.7. CRT	4	7.2. FracBinarySearch	17
2.8. DivModSum	4	7.3. MaximalCliques	17
2.9. FST	4	7.4. MaximumClique	18
2.10. BerlekampMassey	4	7.5. Frievalds	18
2.11. Lagrange	5	8. Hình	18
2.12. SumPowerPoly	5	8.1. Point	18
2.13. XorBasis	5	8.2. TrianglePointCount	19
3. Cấu trúc dữ liệu	6	8.3. SideOf	19
3.1. RMQ	6	8.4. ClosestPair	19
3.2. HLD	6	8.5. ConvexHull	19
3.3. AuxiliaryTree	7	8.6. OnSegment	20
3.4. PersistentSegTree	7	8.7. SegmentIntersection	20
3.5. DSURollback	8	8.8. LineDistance	20
3.6. LineContainer	8	8.9. LineIntersection	20
3.7. SWAD	8	8.10. LineHullIntersection	20
3.8. Mo	9	8.11. LineProjectionReflection	21
4. Đồ thị	9	8.12. CircleLine	21
4.1. SCC	9	8.13. CircleIntersection	21
4.2. EulerWalk	9	8.14. CircleTangents	21
4.3. EnumTriangles	9	8.15. Circumcircle	21
4.4. Dinic	10	8.16. MinimumEnclosingCircle	21
4.5. HopcroftKarp	10	8.17. CirclePolygonIntersection	22
4.6. GeneralMatching	11	8.18. InsidePolygon	22
4.7. MinAssignment	11	8.19. PolygonCenter	22
4.8. CentroidDecomposition	12	8.20. PolygonArea	22
4.9. 2SAT	12	8.21. PointInsideHull	22
4.10. Dominator	13	8.22. HullDiameter	22
		8.23. Minkowski	23

1. Notes

1.1. Chuỗi Taylor

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$$

$$= f(a) + f'(a)(x-a) + \frac{f''(a)}{2!} (x-a)^2 + \dots$$

Với α là số thực bất kì, k là số nguyên:

$$\binom{\alpha}{k} = \begin{cases} 1 & \text{nếu } k = 0 \\ \frac{\alpha(\alpha-1)\dots(\alpha-k+1)}{k!} & \text{nếu } k > 0 \end{cases}$$

1.2. Catalan

$$C_n = \frac{1}{n+1} \binom{2n}{n}, C_{n+1} = \sum_{i=0}^n C_i C_{n-i}$$

1.3. Lucas

Với $n = n_k p^k + n_{k-1} p^{k-1} + \dots + n_0$ và $m = m_k p^k + m_{k-1} p^{k-1} + \dots + m_0$. Ta có $\binom{n}{m} = \prod_{i=0}^k \binom{n_i}{m_i} \pmod p$.

1.4. Bổ đề Burnside

Đặt G là nhóm hữu hạn tác động lên tập X . Với mỗi $g \in G$, gọi X^g là tập các điểm bất định bởi g ($\{x \in X \mid g.x = x\}$). Số quỹ đạo có thể có là:

$$\left| \frac{X}{G} \right| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

1.5. Định lý Pick

Cho một đa giác có các điểm nguyên. Gọi i là số điểm nguyên nằm trong đa giác, và b là số điểm nguyên nằm trên cạnh. Diện tích của đa giác là: $A = i + \frac{b}{2} - 1$.

1.6. Frobenius

$$g = \frac{F+1}{2}, \quad F = (n-1)P - \sum_{\{i=1\}}^n \left(\frac{P}{p_i} \right)$$

1.7. Super interpretation of k th powers

The square of the size of a set is equal to the number of ordered pairs of elements in the set. So we iterate over pairs and for each we compute the contribution to the answer.

Similarly, the k -th power is equal to the number of sequences (tuples) of length k .

$$E(X^2) = E(\# \text{ordered pairs}), E(X^k) = E(\# \text{ordered tuples})$$

If you want to maintain the sum of k -th powers, it might help to also maintain the sum of smaller powers. For example, if the sum of 0-th, 1-th and 2-nd powers is S_0 , S_1 and S_2 , and we increase all elements by x , the new sums are S_0 , $S_1 + S_0 x$ and $S_2 + 2xS_1 + x^2 S_0$.

1.8. Nhận xét

- Cho 2 chuỗi S, T . Số chuỗi phân biệt của $\text{prefix}(S) + \text{suffix}(T) = |S| * |T|$ - số kí tự giống nhau của S và T , không tính S_0 và T_n .

2. Toán

2.1. ModInt

Biểu diễn số nguyên modulo sử dụng Montgomery.

```
#pragma once

template <int mod>
struct modint {
    using M = modint;
    static_assert(mod > 0 && mod <= 2147483647);
    static constexpr int modulo = mod;
    static constexpr u32 r1 = []() {
        u32 r1 = mod;
        for (int i = 0; i < 5; ++i) r1 *= 2 - mod * r1;
        return -r1;
    }();
    static constexpr u32 r2 = -u64(mod) % mod;
    static u32 reduce(u64 x) {
        u32 y = u32(x) * r1, r = (x + u64(y) * mod) >> 32;
        return r >= mod ? r - mod : r;
    }
    u32 x;
    modint() : x(0) {}
    modint(i64 x) : x(reduce(u64(x % mod + mod) * r2)) {}
    M& operator+=(const M& a) {
        if ((x += a.x) >= mod) x -= mod;
        return *this;
    }
```

```
}
M& operator-=(const M& a) {
    if ((x += mod - a.x) >= mod) x -= mod;
    return *this;
}
M& operator*=(const M& a) {
    x = reduce(u64(x) * a.x);
    return *this;
}
M& operator/=(const M& a) { return *this *= a.inv(); }
M operator-(const M& a) const { return M(0) - *this; }
M operator+(const M& a) const { return M(*this) += a; }
M operator-(const M& a) const { return M(*this) -= a; }
M operator*(const M& a) const { return M(*this) *= a; }
M operator/(const M& a) const { return M(*this) /= a; }
bool operator==(const M& a) const { return x == a.x; }
bool operator!=(const M& a) const { return x != a.x; }
M pow(u64 k) const {
    M res(1), b = *this;
    while (k) {
        if (k & 1) res *= b;
        b *= b, k >>= 1;
    }
    return res;
}
M inv() const { return pow(mod - 2); }
friend ostream& operator<<(ostream& os, const M& a) {
    return os << reduce(a.x);
}
friend istream& operator>>(istream& is, M& a) {
    i64 v;
    is >> v;
    a = M(v);
    return is;
}
```

```
};

u64 modmul(u64 x, u64 y, u64 m) { return u128(x) * y % m; }
u64 modpow(u64 x, u64 k, u64 m) {
    u64 res = 1;
    while (k) {
        if (k & 1) res = modmul(res, x, m);
        x = modmul(x, x, m);
        k >>= 1;
    }
    return res;
}
```

2.2. MillerRabin

Kiểm tra số nguyên tố nhanh, **chắc chắn** đúng trong unsigned long long.

```
bool isPrime(u64 n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    u64 A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n - 1), d = n >> s;
    for (u64 a : A) { // ^ count trailing zeroes
        u64 p = modpow(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--) p = modmul(p, p, n);
        if (p != n - 1 && i != s) return 0;
    }
    return 1;
}
```

2.3. Matrix

Ma trận vuông, hỗ trợ nhân và lũy thừa.

```
template <class T>
struct Matrix {
    using vec = vector<T>;
    int n;
    vector<vec> a;
    Matrix(int n = 0) : n(n), a(n, vec(n, 0)) {}
    Matrix(const vector<vec>& a) : n(sz(a)), a(a) {}
```

```
vec& operator[](int i) { return a[i]; }
const vec& operator[](int i) const { return a[i]; }
Matrix operator*(const Matrix& b) const {
    Matrix res(n);
    for (int i = 0; i < n; ++i)
        for (int k = 0; k < n; ++k)
            for (int j = 0; j < n; ++j)
                res[i][j] += a[i][k] * b[k][j];
    return res;
}
Matrix operator^(u64 k) const {
    Matrix res(n), b = *this;
    for (int i = 0; i < n; ++i) res[i][i] = 1;
    while (k) {
        if (k & 1) res = res * b;
        b = b * b, k >>= 1;
    }
    return res;
}
vec operator*(const vec& v) const { // b(v)
    vec c(n);
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) c[i] += a[i][j] * v[j];
    return c;
}
};
```

2.4. ModLog

Tìm $x > 0$ nhỏ nhất sao cho $a^x = b \bmod m$, hoặc -1 .

$\text{modLog}(a, 1, m)$ trả về order của a trong $\mathbb{Z}_m^*$. Độ phức tạp $O(\sqrt{m})$.

```
i64 modLog(i64 a, i64 b, i64 m) {
    i64 n = (i64) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<i64, i64> A;
    while (j <= n && (e = f = e * a % m) != b % m) A[e * b % m] = j++;
    if (e == b % m) return j;
    if (gcd(m, e) == gcd(m, b)) {
        for (int i = 2; i < n + 2; ++i) {
```

```
        if (A.count(e = e * f % m)) return n * i - A[e];
    }
    return -1;
}
```

2.5. ModSqrt

Tìm căn bậc hai modulo p nguyên tố trong trung bình $O(\log p)$.

```
i64 modsqrt(i64 a, i64 p) {
    a %= p;
    if (a < 0) a += p;
    if (a == 0) return 0;

    if (modpow(a, (p - 1) / 2, p) != 1) return -1;
    if (p % 4 == 3) return modpow(a, (p + 1) / 4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    i64 s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0) ++r, s /= 2;
    /// find a non-square mod p
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    i64 x = modpow(a, (s + 1) / 2, p);
    i64 b = modpow(a, s, p), g = modpow(n, s, p);
    for (; r = m) {
        i64 t = b;
        for (m = 0; m < r && t != 1; ++m) t = t * t % p;
        if (m == 0) return x;
        i64 gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

2.6. Factor

Tìm một ước của n nhanh trong $O(\sqrt[4]{n} \log n)$. Phân tích đệ quy n thành thừa số nguyên tố.

```
#include "ModInt.h"
#include "MillerRabin.h"

u64 pollard(u64 n) {
    u64 x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](u64 x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x, y) - min(x, y), n)))
            prd = q;
        x = f(x), y = f(f(y));
    }
    return gcd(prd, n);
}

vector<u64> factor(u64 n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    u64 x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

2.7. CRT

Duy trì hệ phương trình đồng dư.

```
template <class T>
struct CRT {
    T res;
    CRT() { res = 0, prd = 1; }
    // Add condition: res % p == r
    void add(T p, T r) {
        res += mul(r - res % p + p, euclid(prd, p).first
            + p, p) * prd;
        prd *= p;
        if (res >= prd) res -= prd;
    }

private:
    T prd;
    T mul(T a, T b, T p) {
```

```
a %= p, b %= p;
T q = (T) ((ld) a * b / p);
T r = a * b - q * p;
while (r < 0) r += p;
while (r >= p) r -= p;
return r;
}

pair<T, T> euclid(T a, T b) {
    if (!b) return make_pair(1, 0);
    pair<T, T> r = euclid(b, a % b);
    return make_pair(r.second, r.first - a / b *
        r.second);
}
};
```

2.8. DivModSum

Tính $\sum_{i=0}^{n-1} \frac{a+i \times d}{m}$ và $\sum_{i=0}^{n-1} (a + i \times d) \bmod m$. Độ phức tạp $O(\log N)$

```
i64 sumsq(i64 to) { return to / 2 * ((to - 1) |
1); }

// sum( (a + d*i) / m ) for i in [0, n-1]
i64 divsum(i64 a, i64 d, i64 m, i64 n) {
    i64 res = d / m * sumsq(n) + a / m * n;
    d %= m, a %= m;
    if (!d) return res;
    i64 to = (n * d + a) / m;
    return res + (n - 1) * to - divsum(m - 1 - a, m, d,
        to);
}

// sum( (a + d*i) % m ) for i in [0, n-1]
i64 modsum(i64 a, i64 d, i64 m, i64 n) {
    a = ((a % m) + m) % m, d = ((d % m) + m) % m;
    return n * a + d * sumsq(n) - m * divsum(a, d, m,
        n);
}
```

2.9. FST

Tính tích chập AND, OR, XOR.

```
template <class T>
```

```
void FST(vector<T>& a, bool inv, string type) {
    for (int n = sz(a), step = 1; step < n; step *= 2)
    {
        for (int i = 0; i < n; i += 2 * step)
            for (int j = i; j < i + step; ++j) {
                T &u = a[j], &v = a[j + step];
                if (type == "and")
                    tie(u, v) = inv ? tuple{v - u, u} :
                        tuple{v, u + v};
                else if (type == "or")
                    tie(u, v) = inv ? tuple{v, u - v} : tuple{u
                        + v, u};
                else if (type == "xor")
                    tie(u, v) = tuple{u + v, u - v};
            }
    }
    if (inv && type == "xor")
        for (T& x : a) x /= sz(a);
}

template <class T>
vector<T> conv(vector<T> a, vector<T> b, string type)
{
    FST(a, 0, type);
    FST(b, 0, type);
    for (int i = 0; i < sz(a); ++i) a[i] *= b[i];
    FST(a, 1, type);
    return a;
}
```

2.10. BerlekampMassey

Phục hồi một dãy truy hồi cấp n từ $2n$ số hạng đầu tiên trong $O(n^2)$.

```
template <class Fp>
vector<Fp> BerlekampMassey(const vector<Fp>& s) {
    if (s.empty()) return {};
    int n = sz(s), L = 0, m = 0;
    vector<Fp> C(n), B(n), T;
    C[0] = B[0] = 1;
    Fp b = 1;
    for (int i = 0; i < n; ++i) {
        ++m;
```

```

Fp d = s[i];
for (int j = 1; j <= L; ++j) d += C[j] * s[i - j];
if (d == 0) continue;
T = C;
Fp coeff = d / b;
for (int j = m; j < n; ++j) C[j] -= coeff * B[j - m];
if (2 * L > i) continue;
L = i + 1 - L, B = T, b = d, m = 0;
}
C.resize(L + 1), C.erase(C.begin());
for (Fp& x : C) x = -x;
return C;
}

```

2.11. Lagrange

Tìm đa thức bậc $n - 1$ qua n điểm trong $O(n^2)$. Vẫn đúng trong trường modulo.

```

typedef vector<ld> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    for (int k = 0; k < n - 1; ++k) {
        for (int i = k + 1; i < n; ++i) {
            y[i] = (y[i] - y[k]) / (x[i] - x[k]);
        }
    };
    ld last = 0;
    temp[0] = 1;
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) {
            res[i] += y[k] * temp[i];
            swap(last, temp[i]);
            temp[i] -= last * x[k];
        }
    }
    return res;
}

```

2.12. SumPowerPoly

```

#include "ModInt.h"
using Fp = modint<998244353>;

vector<Fp> fac, invFac;
void prepareFac(int n) {
    fac.resize(n + 1);
    invFac.resize(n + 1);
    fac[0] = 1;
    for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i;
    invFac[n] = fac[n].inv();
    for (int i = n; i >= 1; --i) invFac[i - 1] = invFac[i] * i;
}

// Lagrange interpolation [0,...,n-1] in O(n)
Fp interpolate(const vector<Fp>& y, i64 n) {
    int k = sz(y) - 1;
    if (n <= k) return y[n];
    vector<Fp> pre(k + 1), suf(k + 1);
    pre[0] = suf[k] = 1;
    for (int i = 0; i < k; ++i) pre[i + 1] = pre[i] * (n - i);
    for (int i = k; i > 0; --i) suf[i - 1] = suf[i] * (n - i);
    Fp ans = 0;
    for (int i = 0; i <= k; ++i) {
        Fp val = pre[i] * suf[i] * y[i] * invFac[i] * invFac[k - i];
        if ((k - i) & 1) ans -= val;
        else ans += val;
    }
    return ans;
}

// C = sum_{i=0->inf} r^i * fs[i] (r != 1)
Fp sumPolyLimit(Fp r, const vector<Fp>& fs) {
    int d = fs.size() - 1;
    if (r.x == 0) return fs[0];
    vector<Fp> rr(d + 1);
    rr[0] = 1;
    for (int i = 1; i <= d; ++i) rr[i] = rr[i - 1] * r;
}

```

```

Fp ans = 0, S = 0;
for (int i = 0; i <= d; ++i) {
    S += rr[i] * fs[i];
    Fp term = invFac[d - i] * invFac[i + 1] * rr[d - i] * S;
    if ((d - i) & 1) ans -= term;
    else ans += term;
}
return ans * fac[d + 1] / (Fp(1) - r).pow(d + 1);
}

// Sum_{i=0->n-1} r^i * fs[i]
Fp sumPoly(Fp r, const vector<Fp>& fs, u64 n) {
    if (n == 0) return 0;
    if (r == 0) return fs[0];
    int d = sz(fs) - 1;
    if (r == 1) {
        vector<Fp> S(d + 2);
        S[0] = 0;
        for (int i = 0; i <= d; ++i) S[i + 1] = S[i] + fs[i];
        return interpolate(S, n);
    }
    Fp C = sumPolyLimit(r, fs), S_curr = 0, rp = 1, rip = 1, ri = r.inv();
    vector<Fp> g(d + 1);
    for (int k = 0; k <= d; ++k) {
        g[k] = (S_curr - C) * rip;
        S_curr += rp * fs[k], rp *= r, rip *= ri;
    }
    return C + r.pow(n) * interpolate(g, n);
}

```

2.13. XorBasis

```

struct XorBasis {
    vector<i64> b;
    XorBasis() {}
    void add(int x) {
        x = this->sift(x);
        if (x != 0) this->b.pb(x);
    }
}

```

```

i64 sift(i64 x) const {
    for (i64 b : this->b) {
        x = min(x, x ^ b);
        if (x == 0) return 0;
    }
    return x;
}

bool is_indep(i64 x) const {
    return this->sift(x) != 0;
}

vector<i64> basis() const {
    return this->b;
}
};

vector<i64> XorInter(const vector<i64>& u, const
vector<i64>& v) {
    XorBasis X;
    for (auto x : u) X.add(x);
    vector<pair<i64, i64>> basis;
    XorBasis inter;
    for (auto x : v) {
        auto y = X.sift(x), pu = y ^ x, sy = y;
        for (auto v : basis) {
            i64 tmp = sy ^ v.second;
            if (tmp < sy) {
                sy = tmp;
                pu ^= v.first;
            }
        }
        if (sy != 0) {
            basis.pb({pu, sy});
        } else {
            inter.add(pu);
        }
    }
    return inter.basis();
}

```

3. Cấu trúc dữ liệu

3.1. RMQ

```

template <class T, class F>
struct RMQ {
    vector<vector<T>> jmp;
    const F f;
    RMQ(const vector<T>& V, F f) : jmp(1, V), f(f) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2,
            ++k) {
            jmp.pb(sz(V) - pw * 2 + 1);
            for (int j = 0; j < sz(jmp[k]); ++j) jmp[k][j]
                = f(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    // [a, b)
    T query(int a, int b) {
        assert(a < b);
        int dep = 31 - __builtin_clz(b - a);
        return f(jmp[dep][a], jmp[dep][b - (1 << dep)]);
    }
};

```

3.2. HLD

HLD cho phép truy vấn cả đường đi và cây con trong $O(\log N)$.

1. pos(x): trả về vị trí của đỉnh x trong quá trình duyệt DFS.
2. subtree(x): trả về đoạn [l, r) tương ứng với cây con của x.
3. path_query(a, b, f): phân hoạch đường đi từ a đến b thành các đoạn liên tiếp trong mảng DFS. Hàm f có thể được dùng để truy vấn/cập nhật Segment Tree.

```

template <class G>
struct HLD {
private:
    void dfs_sz(int cur) {
        size[cur] = 1;
        for (auto& dst : g[cur]) {
            if (dst == par[cur]) {
                if (g[cur].size() >= 2 && int(dst) ==
                    int(g[cur][0]))
                    swap(g[cur][0], g[cur][1]);
                else
                    continue;
            }
            depth[dst] = depth[cur] + 1;
        }
    }
};

```

```

    par[dst] = cur;
    dfs_sz(dst);
    size[cur] += size[dst];
    if (size[dst] > size[g[cur][0]]) swap(dst,
        g[cur][0]);
}

void dfs_hld(int cur) {
    down[cur] = id++;
    for (auto dst : g[cur]) {
        if (dst == par[cur]) continue;
        nxt[dst] = (int(dst) == int(g[cur][0]) ?
            nxt[cur] : int(dst));
        dfs_hld(dst);
    }
    up[cur] = id;
}

// [u, v)
vector<pii> ascend(int u, int v) const {
    vector<pii> res;
    while (nxt[u] != nxt[v]) res.pb(down[u],
        down[nxt[u]]), u = par[nxt[u]];
    if (u != v) res.pb(down[u], down[v] + 1);
    return res;
}

// (u, v)
vector<pii> descend(int u, int v) const {
    if (u == v) return {};
    if (nxt[u] == nxt[v]) return {{down[u] + 1,
        down[v]}};
    auto res = descend(u, par[nxt[v]]);
    res.pb(down[nxt[v]], down[v]);
    return res;
}

public:
    G& g;
    int root, id;
    vi size, depth, down, up, nxt, par;
};

```

```

HLD(G& g, int root = 0) : g(g), root(root), id(0),
size(sz(g), 0), depth(sz(g), 0), down(sz(g), -1),
up(sz(g), -1), nxt(sz(g), root), par(sz(g), root) {
    dfs_sz(root);
    dfs_hld(root);
}

pii idx(int i) const { return make_pair(down[i],
up[i]); }

template <class F>
void path_query(int u, int v, bool vertex, const F&
f) {
    int l = lca(u, v);
    for (auto&& [a, b] : ascend(u, l)) {
        int s = a + 1, t = b;
        s > t ? f(t, s) : f(s, t);
    }
    if (vertex) f(down[l], down[l] + 1);
    for (auto&& [a, b] : descend(l, v)) {
        int s = a, t = b + 1;
        s > t ? f(t, s) : f(s, t);
    }
}

template <class F>
void path_noncommutative_query(int u, int v, bool
vertex, const F& f) {
    int l = lca(u, v);
    for (auto&& [a, b] : ascend(u, l)) f(a + 1, b);
    if (vertex) f(down[l], down[l] + 1);
    for (auto&& [a, b] : descend(l, v)) f(a, b + 1);
}

template <class F>
void subtree_query(int u, bool vertex, const F& f)
{
    f(down[u] + int(!vertex), up[u]);
}

int lca(int a, int b) {
    while (nxt[a] != nxt[b]) {
        if (down[a] < down[b]) swap(a, b);

```

```

        a = par[nxt[a]];
    }
    return depth[a] < depth[b] ? a : b;
}

int dist(int a, int b) { return depth[a] + depth[b]
- depth[lca(a, b)] * 2; }
};

```

3.3. AuxiliaryTree

```

#include "HLD.h"

template <class G>
struct AuxiliaryTree {
    G g;
    HLD<G> hld;
    AuxiliaryTree(const G& g, int root = 0) : g(g),
hld(g, root) {}
    vi get(vi ps) {
        if (ps.empty()) return {};
        auto comp = [&](int i, int j) { return
hld.down[i] < hld.down[j]; };
        sort(all(ps), comp);
        for (int i = 0, ie = sz(ps); i + 1 < ie; i++) {
            ps.pb(hld.lca(ps[i], ps[i + 1]));
        }
        sort(all(ps), comp);
        ps.erase(unique(all(ps)), end(ps));
        vector<vi> aux(sz(ps));
        vi rs;
        rs.pb(0); // root ?
        for (int i = 1; i < sz(ps); i++) {
            int l = hld.lca(ps[rs.back()], ps[i]);
            while (ps[rs.back()] != l) rs.pop_back();
            aux[rs.back()].pb(i), rs.pb(i);
        }
        return aux;
    }
};

```

3.4. PersistentSegTree

```

/*
Persistent + Dynamic Segment Tree that supports
Monoid operation.
Tested on https://cses.fi/problemset/task/1737/
*/

template <class T, class F>
struct PST {
    struct Node {
        T v;
        Node *l = nullptr, *r = nullptr;
        Node(T v) : v(v) {}
    };
    int n;
    const F f;
    const T I;
    PST(int n, F f, const T& I) : n(n), f(f), I(I) {}
    T get_val(Node* u) const { return u ? u->v : I; }

    Node* apply(Node* prev, int L, int R, int pos,
const T& nv) {
        Node* u = new Node(prev ? prev->v : I);
        if (prev) u->l = prev->l, u->r = prev->r;
        if (L == R) {
            u->v = nv;
            return u;
        }
        int M = (L + R) >> 1;
        if (pos <= M) {
            u->l = apply(u->l, L, M, pos, nv);
        } else {
            u->r = apply(u->r, M + 1, R, pos, nv);
        }
        u->v = f(get_val(u->l), get_val(u->r));
        return u;
    }

    // [ql, qr] inclusive
    T query(Node* u, int L, int R, int ql, int qr)
const {
        if (!u || qr < L || R < ql) return I;
        if (ql <= L && R <= qr) return u->v;
        int M = (L + R) >> 1;

```

```

    return f(query(u->l, L, M, ql, qr), query(u->r, M
+ 1, R, ql, qr));
}
};

```

3.5. DSURollback

```

struct DSURollback {
    int n;
    vector<int> p;
    vector<pair<int*, int>> his;
    DSURollback(int n) : n(n), p(n, -1) {}
    int root(int s) {
        while (p[s] >= 0) s = p[s];
        return s;
    }
    void merge(int a, int b) {
        a = root(a);
        b = root(b);
        if (a == b) return;
        if (p[a] < p[b]) swap(a, b);
        his.eb(&p[a], p[a]), his.eb(&p[b], p[b]),
        his.eb(&n, n);
        n--, p[b] += p[a], p[a] = b;
    }
    void undo(int cnt) {
        while (cnt--) {
            auto [a, b] = his.back();
            his.pop_back();
            *a = b;
        }
    }
};

```

3.6. LineContainer

Duy trì tập các đường thẳng dạng $y = kx + m$ và truy vấn giá trị **lớn nhất** tại điểm x . Nếu muốn tìm giá trị nhỏ nhất, đổi dấu k, m và kết quả truy vấn.

```

struct Line {
    mutable i64 k, m, p;

```

```

    bool operator<(const Line& o) const { return k <
o.k; }
    bool operator<(i64 x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for lds, use inf = 1/.0, div(a,b) = a/b)
    static const i64 inf = LLONG_MAX;
    i64 div(i64 a, i64 b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k)
            x->p = x->m > y->m ? inf : -inf;
        else
            x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(i64 k, i64 m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y =
erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    i64 query(i64 x) { // return max
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

```

3.7. SWAD

```

template <class T, class F>
struct SWAD {
    vector<T> a0, a1, r0, r1;
    F f;
    T I;
    SWAD(F f, T i) : f(f), I(i) {}

```

```

private:
    T get0() const { return r0.empty() ? I :
r0.back(); }
    T get1() const { return r1.empty() ? I :
r1.back(); }

    void push0(const T& x) {
        a0.eb(x, r0.eb(f(x, get0())));
    }
    void push1(const T& x) {
        a1.eb(x, r1.eb(f(get1(), x)));
    }
    void rebalance() {
        int n = a0.size() + a1.size();
        int s0 = n / 2 + (a0.empty() ? n % 2 : 0);
        vector<T> a{a0};
        reverse(all(a));
        copy(all(a1), back_inserter(a));
        a0.clear(), r0.clear(), a1.clear(), r1.clear();
        for (int i = s0 - 1; i >= 0; i--) push0(a[i]);
        for (int i = s0; i < n; i++) push1(a[i]);
    }

public:
    void push_front(const T& t) { push0(t); }
    void push_back(const T& t) { push1(t); }
    T front() const { return a0.empty() ? a1.front() :
a0.back(); }
    T back() const { return a1.empty() ? a0.front() :
a1.back(); }
    void pop_front() {
        if (a0.empty()) rebalance();
        assert(!a0.empty());
        a0.pop_back(), r0.pop_back();
    }
    void pop_back() {
        if (a1.empty()) rebalance();
        assert(!a1.empty());
        a1.pop_back(), r1.pop_back();
    }
    T query() { return f(get0(), get1()); }
};

```

3.8. Mo

```
const int sz = 850; // should be sqrt(3/2 * N)
struct Query {
    int l, r, idx;
    bool operator<(const Query& o) {
        if (l / sz != o.l / sz) return l / sz < o.l / sz;
        else {
            if ((l / sz) & 1) return r / sz < o.r / sz;
            else return r / sz > o.r / sz;
        }
    };
};
// handle [l, r] inclusive:
// int pl = 0, pr = -1;
// for (auto [l, r, idx] : qry) {
//     while (pr < r) add(x[++pr]);
//     while (l < pl) add(x[--pl]);
//     while (pl < l) rem(x[pl++]);
//     while (r < pr) rem(x[pr--]);
//     ans[idx] = res;
// }
```

4. Đồ thị

4.1. SCC

```
template <class G>
struct SCC {
public:
    vector<vi> dag;
    SCC(G& g) : g(g), used(sz(g), 0) { build(); }
    int operator[](int k) { return comp[k]; }
    vi& belong(int i) { return blng[i]; }

private:
    const G& g;
    vector<vi> rg;
    vi comp, ord;
    vector<bool> used;
    vector<vi> blng;
```

```
void dfs(int idx) {
    if (used[idx]) return;
    used[idx] = true;
    for (auto to : g[idx]) dfs(to);
    ord.eb(idx);
}

void rdfs(int idx, int cnt) {
    if (comp[idx] != -1) return;
    comp[idx] = cnt;
    for (int to : rg[idx]) rdfs(to, cnt);
}

void build() {
    for (int i = 0; i < sz(g); i++) dfs(i);
    reverse(all(ord));
    used.clear(), used.shrink_to_fit();
    comp.resize(sz(g), -1);
    rg.resize(sz(g));
    for (int i = 0; i < sz(g); i++) {
        for (auto e : g[i]) {
            rg[e].emplace_back(i);
        }
    }
    int ptr = 0;
    for (int i : ord)
        if (comp[i] == -1) rdfs(i, ptr), ptr++;
    rg.clear(), rg.shrink_to_fit();
    ord.clear(), ord.shrink_to_fit();
    dag.resize(ptr), blng.resize(ptr);
    for (int i = 0; i < (int) sz(g); i++) {
        blng[comp[i]].eb(i);
        for (auto& to : g[i]) {
            int x = comp[i], y = comp[to];
            if (x == y) continue;
            dag[x].eb(y);
        }
    }
}

};
```

4.2. EulerWalk

```
pair<vi, vi> EulerWalk(int n, vector<vector<pii>>& adj, int m, bool dir, bool cyc) {
    vi D(n), ptr(n), used(m), nodes, edges;
    vector<pii> st;
    int src = 0, bad = 0;
    for (int i = 0; i < n; ++i) {
        if (dir) for (auto& p : adj[i]) D[i]++, D[p.first]--;
        else D[i] = sz(adj[i]) & 1;
    }
    for (int i = 0; i < n; ++i) {
        if (sz(adj[i]) && adj[src].empty()) src = i;
        if (D[i]) {
            bad++;
            if ((dir && D[i] > 0) || (!dir)) src = i;
        }
    }
    if (bad > 2 || (cyc && bad) || (dir && bad && D[src] != 1)) return {};
    st.pb({src, -1});
    while (!st.empty()) {
        int u = st.back().first;
        if (ptr[u] < sz(adj[u])) {
            auto [v, id] = adj[u][ptr[u]++];
            if (!used[id]) used[id] = 1, st.pb({v, id});
        } else {
            auto [v, id] = st.back();
            st.pop_back();
            nodes.pb(v);
            if (id != -1) edges.pb(id);
        }
    }
    if (sz(edges) != m) return {};
    reverse(all(nodes)), reverse(all(edges));
    return {nodes, edges};
}
```

4.3. EnumTriangles

Duyệt qua tất cả tam giác của đồ thị trong $O(M^{\frac{4}{3}})$

```
template <class F>
```

```

void EnumTriangles(int n, const vector<pii>& ed, F f)
{ // 0-indexed graph
  vi deg(n);
  for (auto [u, v] : ed) ++deg[u], ++deg[v];
  vector<vi> g(n); // directed
  for (auto& e : ed) {
    auto [u, v] = e;
    if (tie(deg[u], u) > tie(deg[v], v)) swap(u, v);
    g[u].eb(v);
  }
  vector<bool> adj(n);
  for (auto& [u, v] : ed) {
    for (auto nu : g[u]) adj[nu] = true;
    for (auto nv : g[v]) {
      if (adj[nv]) f(u, v, nv);
    }
    for (auto nu : g[u]) adj[nu] = false;
  }
}

```

4.4. Dinic

Tìm luồng cực đại bằng Dinic trong với U là luồng tối đa trên một cạnh. Nếu mọi cạnh đều có luồng 1 thì độ phức tạp là $O(\min(E^{\frac{2}{3}}, V^{\frac{1}{2}})E)$.

Với bài cặp ghép 2 phía, độ phức tạp là $O(E\sqrt{V})$.

```

struct Dinic {
  struct Edge {
    int to, rev;
    i64 c, oc;
    i64 flow() { return max(oc - c, i64(0)); } // if
    you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, i64 c, i64 rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap,
    rcap});
  }
  i64 dfs(int v, int t, i64 f) {

```

```

    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (i64 p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  i64 calc(int s, int t) {
    i64 flow = 0;
    q[0] = s;
    // 'int L=30' maybe faster for random data
    for (int L = 0; L < 31; ++L) {
      do {
        lvl = ptr = vi(sz(q));
        int qi = 0, qe = lvl[s] = 1;
        while (qi < qe && !lvl[t]) {
          int v = q[qi++];
          for (Edge e : adj[v])
            if (!lvl[e.to] && e.c >> (30 - L))
              q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
        }
        while (i64 p = dfs(s, t, LLONG_MAX)) flow += p;
      } while (lvl[t]);
    }
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

```

4.5. HopcroftKarp

Cặp ghép cực đại trên đồ thị 2 phía trong $O(E\sqrt{V})$. 0-indexed.

Định lý König: Trong đồ thị 2 phía, MIS = N - cặp ghép cực đại.

Cách dùng: vi btoa(m, -1); hopcroftKarp(g, btoa);

```

bool dfs(int a, int L, vector<vi>& g, vi& btoa,
vi& A, vi& B) {
  if (A[a] != L) return 0;

```

```

  A[a] = -1;
  for (int b : g[a]) {
    if (B[b] == L + 1) {
      B[b] = 0;
      if (btoa[b] == -1 || dfs(btoa[b], L + 1, g,
      btoa, A, B)) return btoa[b] = a, 1;
    }
  }
  return 0;
}

int hopcroftKarp(vector<vi>& g, vi& btoa) {
  int res = 0;
  vi A(g.size()), B(btoa.size()), cur, next;
  for (;;) {
    fill(all(A), 0);
    fill(all(B), 0);
    /// Find the starting nodes for BFS (i.e. layer 0).
    cur.clear();
    for (int a : btoa)
      if (a != -1) A[a] = -1;
    for (int a = 0; a < sz(g); ++a)
      if (A[a] == 0) cur.push_back(a);
    /// Find all layers using bfs.
    for (int lay = 1; lay++;) {
      bool islast = 0;
      next.clear();
      for (int a : cur)
        for (int b : g[a]) {
          if (btoa[b] == -1) {
            B[b] = lay;
            islast = 1;
          } else if (btoa[b] != a && !B[b]) {
            B[b] = lay;
            next.push_back(btoa[b]);
          }
        }
      if (islast) break;
      if (next.empty()) return res;
      for (int a : next) A[a] = lay;
      cur.swap(next);
    }
  }
}

```

```

    }
    /// Use DFS to scan for augmenting paths.
    for (int a = 0; a < sz(g); ++a) res += dfs(a, 0,
        g, btoa, A, B);
    }
}

```

4.6. GeneralMatching

Tìm cặp ghép cực đại trên đồ thị thường trong $O(V^3)$. 0-indexed.

```

vi GeneralMatching(vector<vi>& graph) {
    int n = graph.size(), timer = -1;
    vi mate(n, -1), label(n), parent(n), orig(n),
    aux(n, -1), q;
    auto lca = [&](int x, int y) {
        for (timer++; swap(x, y)) {
            if (x == -1) continue;
            if (aux[x] == timer) return x;
            aux[x] = timer;
            x = (mate[x] == -1 ? -1 :
                orig[parent[mate[x]]]);
        }
    };
    auto blossom = [&](int v, int w, int a) {
        while (orig[v] != a) {
            parent[v] = w;
            w = mate[v];
            if (label[w] == 1) label[w] = 0,
            q.push_back(w);
            orig[v] = orig[w] = a;
            v = parent[w];
        }
    };
    auto augment = [&](int v) {
        while (v != -1) {
            int pv = parent[v], nv = mate[pv];
            mate[v] = pv;
            mate[pv] = v;
            v = nv;
        }
    };
    auto bfs = [&](int root) {

```

```

        fill(label.begin(), label.end(), -1);
        iota(orig.begin(), orig.end(), 0);
        q.clear();
        label[root] = 0;
        q.push_back(root);
        for (int i = 0; i < (int) q.size(); ++i) {
            int v = q[i];
            for (auto x : graph[v]) {
                if (label[x] == -1) {
                    label[x] = 1;
                    parent[x] = v;
                    if (mate[x] == -1)
                        return augment(x, 1);
                    label[mate[x]] = 0;
                    q.push_back(mate[x]);
                } else if (label[x] == 0 && orig[v] !=
                    orig[x]) {
                    int a = lca(orig[v], orig[x]);
                    blossom(x, v, a);
                    blossom(v, x, a);
                }
            }
        }
        return 0;
    };
    /// Time halves if you start with (any) maximal
    matching.
    for (int i = 0; i < n; i++)
        if (mate[i] == -1)
            bfs(i);
    return mate;
}

```

4.7. MinAssignment

Nhanh hơn Hungarian nhiều. Muốn tìm max cost, đặt cost âm. 0-indexed.

```

pair<i64, vector<int>> MinAssignment(const
vector<vector<i64>>& W) {
    int n = W.size(), m = W[0].size(); // assert(n <=
    m);

```

```

    vector<i64> v(m), dist(m); // v:
    potential
    vector<int> L(n, -1), R(m, -1); // matching
    pairs
    vector<int> idx(m), prev(m);
    iota(idx.begin(), idx.end(), 0);

    i64 w, h;
    int j, l, s, t;
    auto reduce = [&]() {
        if (s == t) {
            l = s;
            w = dist[idx[t++]];
            for (int k = t; k < m; ++k) {
                j = idx[k];
                h = dist[j];
                if (h > w) continue;
                if (h < w) t = s, w = h;
                idx[k] = idx[t];
                idx[t++] = j;
            }
            for (int k = s; k < t; ++k) {
                j = idx[k];
                if (R[j] < 0) return 1;
            }
        }
        int q = idx[s++], p = R[q];
        for (int k = t; k < m; ++k) {
            j = idx[k];
            h = W[p][j] - W[p][q] + v[q] - v[j] + w;
            if (h < dist[j]) {
                dist[j] = h;
                prev[j] = p;
                if (h == w) {
                    if (R[j] < 0) return 1;
                    idx[k] = idx[t];
                    idx[t++] = j;
                }
            }
        }
    };
    return 0;
};

```

```

for (int i = 0; i < n; ++i) {
    for (int k = 0; k < m; ++k) dist[k] = W[i][k] -
        v[k], prev[k] = i;
    s = t = 0;
    while (!reduce());
    for (int k = 0; k < l; ++k) v[idx[k]] +=
        dist[idx[k]] - w;
    for (int k = -1; k != i; i) R[j] = k = prev[j],
        swap(j, L[k]);
}
i64 ret = 0;
for (int i = 0; i < n; ++i) ret += W[i][L[i]]; //
(i, L[i]) is a solution
return {ret, L};
}

```

4.8. CentroidDecomposition

```

template <class G> h
struct CentroidDecomposition {
    const G& g;
    vi sub;
    vector<bool> v;
    vector<vi> tree;
    int root;

    CentroidDecomposition(const G& g, int isbuild =
        true) : g(g) {
        sub.resize(g.size(), 0);
        v.resize(g.size(), false);
        if (isbuild) build();
    }

    void build() {
        tree.resize(g.size());
        root = build_dfs(0);
    }

    int get_size(int cur, int par) {
        sub[cur] = 1;
        for (auto& dst : g[cur]) {
            if (dst == par || v[dst]) continue;
            sub[cur] += get_size(dst, cur);

```

```

        }
        return sub[cur];
    }

    int get_centroid(int cur, int par, int mid) {
        for (auto& dst : g[cur]) {
            if (dst == par || v[dst]) continue;
            if (sub[dst] > mid) return get_centroid(dst,
                cur, mid);
        }
        return cur;
    }

    int build_dfs(int cur) {
        int centroid = get_centroid(cur, -1,
            get_size(cur, -1) / 2);
        v[centroid] = true;
        for (auto& dst : g[centroid]) {
            if (!v[dst]) {
                int nxt = build_dfs(dst);
                if (centroid != nxt) tree[centroid].eb(nxt);
            }
        }
        v[centroid] = false;
        return centroid;
    }
};

```

4.9. 2SAT

```

/** h
 * Usage: TwoSat ts(number of boolean variables);
 * ts.either(0, ~3); // Var 0 is true or var 3 is
 * false
 * ts.setValue(2); // Var 2 is true
 * ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and
 * 2 are true
 * ts.solve(); // Returns true iff it is solvable
 * ts.values[0..N-1] holds the assigned values to
 * the vars
 */
struct TwoSat {
    int N;

```

```

    vector<vi> gr;
    vi values; // 0 = false, 1 = true
    TwoSat(int n = 0) : N(n), gr(2 * n) {}
    int addVar() { // (optional)
        gr.eb(), gr.eb();
        return N++;
    }
    void either(int f, int j) {
        f = max(2 * f, -1 - 2 * f), j = max(2 * j, -1 - 2
            * j);
        gr[f].eb(j ^ 1), gr[j].eb(f ^ 1);
    }
    void setValue(int x) { either(x, x); }
    void atMostOne(const vi& li) { // (optional)
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        for (int i = 2; i < sz(li); ++i) {
            int next = addVar();
            either(cur, ~li[i]), either(cur, next),
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }
    vi val, comp, z;
    int time = 0;
    int dfs(int i) {
        int low = val[i] = ++time, x;
        z.push_back(i);
        for (int e : gr[i]) {
            if (!comp[e]) low = min(low, val[e] ? : dfs(e));
        }
        if (low == val[i]) {
            do {
                x = z.back(), z.pop_back(), comp[x] = low;
                if (values[x >> 1] == -1) values[x >> 1] = x
                    & 1;
            } while (x != i);
        }
        return val[i] = low;
    }
    bool solve() {

```

```

values.assign(N, -1);
val.assign(2 * N, 0);
comp = val;
for (int i = 0; i < 2 * N; ++i)
    if (!comp[i]) dfs(i);
for (int i = 0; i < N; ++i)
    if (comp[2 * i] == comp[2 * i + 1]) return 0;
return 1;
}
};

```

4.10. Dominator

Dựng Dominator Tree cho đồ thị có hướng khi đặt gốc là s . u là cha của v nếu mọi đường đi từ s đến v đều phải đi qua u . Độ phức tạp $O(M \log N)$ hằng số thấp.

```

vector<int> DomTree(vector<vector<int>>& graph,
int src) {
    int n = graph.size();
    vector<vector<int>> tree(n), trans(n), buck(n);
    vector<int> semi(n), par(n), dom(n), label(n),
    atob(n, -1), btoa(n, -1), link(n, -1);

    function<int(int, int)> find = [&](int u, int d) {
        if (link[u] == -1) return d ? -1 : u;
        int v = find(link[u], d + 1);
        if (v < 0) return u;
        if (semi[label[link[u]]] < semi[label[u]])
            label[u] = label[link[u]];
        link[u] = v;
        return d ? v : label[u];
    };

    int t = 0;
    function<void(int)> dfs = [&](int u) {
        atob[u] = t;
        btoa[t] = u;
        label[t] = semi[t] = t;
        t++;
        for (auto v : graph[u]) {
            if (atob[v] == -1) dfs(v), par[atob[v]] = atob[u];
            trans[atob[v]].push_back(atob[u]);
        }
    };
}

```

```

}
};
dfs(src);
for (int u = t - 1; u >= 0; --u) {
    for (auto v : trans[u]) semi[u] = min(semi[u],
    semi[find(v, 0)]);
    if (u) buck[semi[u]].push_back(u);
    for (auto w : buck[u]) {
        int v = find(w, 0);
        dom[w] = semi[v] == semi[w] ? semi[w] : v;
    }
    if (u) link[u] = par[u];
}
vector<int> ret(n, -1);
for (int u = 1; u < t; ++u) {
    if (dom[u] != semi[u]) dom[u] = dom[dom[u]];
    ret[btoa[u]] = btoa[dom[u]];
}
return ret;
}
}

```

4.11. GomoryHu

Dựng cây Gomory-Hu của đồ thị luồng trong $N - 1$ lần chạy luồng. Max flow/min cut giữa 2 đỉnh u, v trên đồ thị luồng là trọng số cạnh nhỏ nhất trên đường đi từ u đến v .

```

#include "Dinic.h"
typedef array<i64, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
    vector<Edge> tree;
    vi par(N);
    for (int i = 1; i < N; ++i) {
        Dinic D(N);
        for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        for (int j = i + 1; j < N; ++j)
            if (par[j] == par[i] && D.leftOfMinCut(j))
                par[j] = i;
    }
    return tree;
}

```

```

}

```

4.12. MinCostMaxFlow

Min-cost max-flow. If costs can be negative, call setpi before maxflow, not support negative cycle. To obtain the actual flow, look at positive values only.

Time: $O(FE \log(V))$ where F is max flow. $O(VE)$ for setpi.

```

const i64 INF = numeric_limits<i64>::max() / 4;
struct MCMF {
    struct edge {
        int from, to, rev;
        i64 cap, cost, flow;
    };
    int N;
    vector<vector<edge>> ed;
    vi seen;
    vector<i64> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, i64 cap, i64 cost) {
        if (from == to) return;
        ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0});
        ed[to].push_back(edge{to, from, sz(ed[from]) - 1, 0, -cost, 0});
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0;
        i64 di;

        __gnu_pbds::priority_queue<pair<i64, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({0, s});
    }
}

```

```

while (!q.empty()) {
    s = q.top().second;
    q.pop();
    seen[s] = 1;
    di = dist[s] + pi[s];
    for (edge& e : ed[s]) {
        if (!seen[e.to]) {
            i64 val = di - pi[e.to] + e.cost;
            if (e.cap - e.flow > 0 && val < dist[e.to]) {
                dist[e.to] = val;
                par[e.to] = &e;
                if (its[e.to] == q.end())
                    its[e.to] = q.push({-dist[e.to],
                    e.to});
                else
                    q.modify(its[e.to], {-dist[e.to],
                    e.to});
            }
        }
    }
}

for (int i = 0; i < N; ++i) pi[i] = min(pi[i] +
dist[i], INF);
}

```

```

pair<i64, i64> maxflow(int s, int t) {
    i64 totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        i64 fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    for (int i = 0; i < N; ++i)
        for (edge& e : ed[i]) totcost += e.cost *
        e.flow;
    return {totflow, totcost / 2};
}

```

```

}

// If some costs can be negative, call this before
maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF);
    pi[s] = 0;
    int it = N, ch = 1;
    i64 v;
    while (ch-- && it--) {
        for (int i = 0; i < N; ++i) {
            if (pi[i] != INF) {
                for (edge& e : ed[i]) {
                    if (e.cap) {
                        if ((v = pi[i] + e.cost) < pi[e.to])
                            pi[e.to] = v, ch = 1;
                    }
                }
            }
        }
        assert(it >= 0); // negative cost cycle
    }
}
};

```

4.13. GlobalMinCut

Tìm lát cắt cực tiểu trong đồ thị vô hướng trong $O(V^3)$.

```

pair<int, vi> globalMinCut(vector<vi> mat) {
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    for (int i = 0; i < n; ++i) co[i] = {i};
    for (int ph = 1; ph < n; ++ph) {
        vi w = mat[0];
        size_t s = 0, t = 0;
        for (int it = 0; it < n - ph; ++it) { //  $O(V^2)$ 
            ->  $O(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            for (int i = 0; i < n; ++i) w[i] += mat[t][i];
        }
    }
}

```

```

best = min(best, {w[t] - mat[t][t], co[t]});
co[s].insert(co[s].end(), all(co[t]));
for (int i = 0; i < n; ++i) mat[s][i] += mat[t][i];
for (int i = 0; i < n; ++i) mat[i][s] = mat[s][i];
mat[0][t] = INT_MIN;
}
return best;
}

```

5. Xâu

5.1. KMP

```

vi pi(const string& s) {
    vi p(sz(s));
    for (int i = 1; i < sz(s); ++i) {
        int g = p[i - 1];
        while (g && s[i] != s[g]) g = p[g - 1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}

```

5.2. Z

```

vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    for (int i = 1; i < sz(S); ++i) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] ==
        S[z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}

```

5.3. MinRotation

Tìm cyclic shift của xâu có thứ tự từ điển nhỏ nhất trong $O(n)$.

```

int minRotation(string s) {
    int a = 0, N = sz(s);
    s += s;
    for(int b = 0; b < N; ++b) for(int k = 0; k < N; +
+k) {
        if (a + k == b || s[a + k] < s[b + k]) {
            b += max(0, k - 1);
            break;
        }
        if (s[a + k] > s[b + k]) {
            a = b;
            break;
        }
    }
    return a;
}

```

5.4. SuffixArray

```

struct SuffixArray {
    vector<int> sa, lcp, rank;
    SuffixArray(string s, int lim = 256) {
        int n = s.size() + 1, k = 0, a, b;
        s.push_back(0);
        vector<int> y(n), cnt(max(n, lim));
        sa.resize(n), lcp.resize(n), rank.resize(n);
        for (int i = 0; i < n; ++i) rank[i] = s[i];
        iota(sa.begin(), sa.end(), 0);

        for (int j = 0, p = 0; p < n; j = max(1, j * 2),
lim = p) {
            p = j;
            iota(all(y), n - j);
            for (int i = 0; i < n; ++i)
                if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(cnt), 0);
            for (int i = 0; i < n; ++i) cnt[rank[i]]++;
            for (int i = 1; i < lim; ++i) cnt[i] += cnt[i -
1];
            for (int i = n; i--;) sa[--cnt[rank[y[i]]]] =
y[i];
            swap(rank, y), p = 1, rank[sa[0]] = 0;
            for (int i = 1; i < n; ++i) {

```

```

        a = sa[i - 1], b = sa[i];
        int val_a = (a + j < n) ? y[a + j] : -1;
        int val_b = (b + j < n) ? y[b + j] : -1;
        rank[b] = (y[a] == y[b] && val_a == val_b) ?
p - 1 : p++;
    }
}

for (int i = 0; i < n; ++i) rank[sa[i]] = i;
for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
    for (k && k--, j = sa[rank[i] - 1]; s[i + k] ==
s[j + k]; k++);
};

```

6. Quy hoạch động

6.1. CountSubseq

```

#include "Compressor.h"
template <class T, class Fp>
Fp CountSubseq(vector<T> a) {
    a = compressor<T>(a);
    vi last(sz(a) + 1, -1);
    vector<Fp> f(sz(a) + 1);
    f[0] = 1;
    for (int i = 0; i < sz(a); ++i) {
        f[i + 1] = f[i] * 2;
        if (last[a[i]] >= 0) f[i + 1] -= f[last[a[i]]];
        last[a[i]] = i;
    }
    return f.back() - 1;
}

```

6.2. SOSDP

Toàn bộ implementation SOS DP của VNOI.

```

// SOS xuôi - z(f(s))
for (int k = 0; k < n; k++)
    for (int mask = 0; mask < (1 << n); mask++)

```

```

        if (mask & (1 << k)) dp[mask] += dp[mask ^ (1 <<
k)];

// Hàm đảo dấu - o(f(s))
for (int mask = 0; mask < (1 << n); mask++)
    sos[mask] = f[mask] * (__builtin_parity(mask) ?
-1 : 1);

// SOS ngược - mu(f(s)) = ozo(f(s))
// z(mu(f)) = mu(z(f)) = f

// SOS truy hồi chính nó: f(S) = h(sum f(T), T subset
S) + a(S)
for (int mask = 0; mask < (1 << n); mask++) {
    dp[mask][0] = 0; // trường hợp cơ sở
    for (int k = 1; k <= n; k++) {
        // tính Sum over Proper Subset
        if (mask & (1 << (k - 1))) {
            int sub = mask ^ (1 << (k - 1));
            dp[mask][k] = dp[mask][k - 1] + dp[sub][k - 1]
+ f[sub];
        } else
            dp[mask][k] = dp[mask][k - 1];
    }
    f[mask] = h(dp[mask][n]) + a[mask]; // tính f
}

/*
SOS 2 hàm gọi chéo nhau
f(S) = h1(sum g(T), T subset S) + a(S)
g(S) = h2(sum f(T), T subseq S) + b(S)
*/
for (int mask = 0; mask < (1 << n); mask++) {
    // tính dpG và f
    for (int k = 1; k <= n; k++) {
        if (mask & (1 << (k - 1))) { // truy hồi theo
kiểu proper subset
            int sub = mask ^ (1 << (k - 1));
            dpG[mask][k] = dpG[mask][k - 1] + dpG[sub][k -
1] + g[sub];
        } else
            dpG[mask][k] = dpG[mask][k - 1];
    }
}

```

```

f[mask] = h1(dpG[mask][n]) + a[mask];

// tính dpF và g
dpF[mask][0] = f[mask];
for (int k = 1; k <= n; k++) {
    if (mask & (1 << (k - 1))) // truy hồi theo kiểu subset
        dpF[mask][k] = dpF[mask][k - 1] + dpF[mask ^ (1 << (k - 1))][k - 1];
    else
        dpF[mask][k] = dpF[mask][k - 1];
}
g[mask] = h2(dpF[mask][n]) + b[mask];
}

// Tính tích chập SOS của 2 hàm f, g
// Make fhat[][] = {0} and ghat[][] = {0}
for (int mask = 0; mask < (1 << N); mask++) {
    fhat[__builtin_popcount(mask)][mask] = f[mask];
    ghat[__builtin_popcount(mask)][mask] = g[mask];
}

// Apply zeta transform on fhat[][] and ghat[][]
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int mask = 0; mask < (1 << N); mask++) {
            if ((mask & (1 << j)) != 0) {
                fhat[i][mask] += fhat[i][mask ^ (1 << j)];
                ghat[i][mask] += ghat[i][mask ^ (1 << j)];
            }
        }
    }
}

// Do the convolution and store into h[][] = {0}
for (int mask = 0; mask < (1 << N); mask++) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j <= i; j++) {
            h[i][mask] += fhat[j][mask] * ghat[i - j][mask];
        }
    }
}

```

```

}

// Apply inverse SOS dp on h[][]
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        for (int mask = 0; mask < (1 << N); mask++) {
            if ((mask & (1 << j)) != 0) {
                h[i][mask] -= h[i][mask ^ (1 << j)];
            }
        }
    }
}

for (int mask = 0; mask < (1 << N); mask++)
    fog[mask] = h[__builtin_popcount(mask)][mask];

```

6.3. 1D1D

Nếu hàm $w(i, j)$ thỏa mãn bất đẳng thức tứ giác: $w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ với mọi $a < b < c < d$, thì ta có thể tính hàm DP 1 chiều: $f(i) = \min_{0 \leq j < i} f(j) + w(j, i)$ trong $O(n \log n)$.

```

// n: số 'phần tử' (tính dp từ 1 đến n)
// cost: hàm tính chi phí cost(j, i)
template <class F>
vector<i64> solve_1d1d(int n, F cost) {
    vector<i64> dp(n + 1);
    dp[0] = 0;
    // tuple: {index nguồn, biên trái, biên phải}
    struct State {
        int u, l, r;
    };
    deque<State> dq;
    dq.push_back({0, 1, n});

    // Hàm tiện ích tính giá trị khi chuyển từ j -> i
    auto val = [&](int j, int i) { return dp[j] + cost(j, i); };

    for (int i = 1; i <= n; ++i) {
        // 1. Lấy giá trị tối ưu cho i từ đầu deque
        dp[i] = val(dq.front().u, i);

        // 2. Cập nhật khoảng cover của phần tử 'đầu'
        if (dq.front().r == i) dq.pop_front();
        else dq.front().l = i + 1;
    }
}

```

```

// 3. Thêm i vào deque làm ứng viên mới
// Bỏ những ứng viên ở cuối nếu i tốt hơn họ ngay tại điểm bắt đầu l
while (!dq.empty() && val(i, dq.back().l) <= val(dq.back().u, dq.back().l)) dq.pop_back();
// Nếu deque rỗng, i thắng tất cả -> cover phần còn lại
if (dq.empty()) {
    dq.push_back({i, i + 1, n});
} else {
    // Nếu i thắng ứng viên cuối tại điểm kết thúc r, ta tìm điểm cắt
    // Nếu không thắng tại r, nghĩa là i không bao giờ vượt qua được ứng viên này -> bỏ qua i
    auto& back = dq.back();
    if (back.r > i && val(i, back.r) < val(back.u, back.r)) {
        int l = back.l, r = back.r, pos = r + 1;
        while (l <= r) {
            int mid = l + (r - l) / 2;
            if (val(i, mid) < val(back.u, mid)) pos = mid, r = mid - 1;
            else l = mid + 1;
        }
        back.r = pos - 1;
        dq.push_back({i, pos, n});
    }
}
return dp;
}

```

6.4. Knuth

Nếu hàm $w(i, j)$ thỏa mãn bất đẳng thức tứ giác: $w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ với mọi $a < b < c < d$, thì ta có thể tính hàm DP: $f(i, j) = \min_{i \leq k < j} f(i, k) + f(k + 1, j) + w(j, i)$ trong $O(n^2)$.

```

// N: số 'phần tử'
// cost(i, j): chi phí gộp đoạn [i, j]
template <class F>
i64 knuth(int N, F cost) {
    const i64 INF = 1e18;
}

```

```

vector<vector<i64>> dp(N + 2, vector<i64>(N + 2, 0));
vector<vector<int>> opt(N + 2, vector<int>(N + 2, 0));
// Khởi tạo base case: đoạn độ dài 1
for (int i = 1; i <= N; ++i) dp[i][i] = cost(i, i), opt[i][i] = i;
// Duyệt theo độ dài len
for (int len = 2; len <= N; ++len) {
    for (int i = 1; i <= N - len + 1; ++i) {
        int j = i + len - 1;
        dp[i][j] = INF;
        i64 c = cost(i, j);
        for (int k = opt[i][j - 1]; k <= min(j - 1, opt[i + 1][j]); ++k) {
            i64 curr = dp[i][k] + dp[k + 1][j] + c;
            if (curr < dp[i][j]) dp[i][j] = curr, opt[i][j] = k;
        }
    }
}
return dp[1][N];
}

```

6.5. DnCDP

Nếu hàm $w(i, j)$ thoả mãn bất đẳng thức tứ giác: $w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ với mọi $a < b < c < d$, thì ta có thể tính hàm DP: $f[i][j] = \min_{k < j} (f[i - 1][k] + w(k + 1, j))$ trong $O(n \log n)$.

```

// N: số'phân tử, K: số'đoạn chia
// cost(l, r): trả về'chi phí đoạn [l, r]
template <class F>
i64 solve_dnc(int N, int K, F cost) {
    const i64 INF = 1e18;
    vector<i64> dp_before(N + 1, INF), dp_cur(N + 1);
    // Khởi tạo tầng k=1
    for (int i = 1; i <= N; ++i) dp_before[i] = cost(1, i);
    // Hàm đệ quy xử'ly từng tầng
    auto compute = [&](auto&& self, int L, int R, int optL, int optR) -> void {
        if (L > R) return;

```

```

int mid = (L + R) / 2;
int best_k = -1;
dp_cur[mid] = INF;
// Giới hạn k: từ optL đến min(mid - 1, optR)
for (int k = optL; k <= min(mid - 1, optR); ++k) {
    i64 val = dp_before[k] + cost(k + 1, mid);
    if (val < dp_cur[mid]) {
        dp_cur[mid] = val;
        best_k = k;
    }
}
self(self, L, mid - 1, optL, best_k);
self(self, mid + 1, R, best_k, optR);
};
for (int k = 2; k <= K; ++k) compute(compute, 1, N, 1, N), dp_before = dp_cur;
return dp_before[N];
}

```

7. Khác

7.1. maxHist

Hình chữ nhật lớn nhất.

```

int maxHist(vector<int>& h) {
    h.push_back(0);
    stack<int> st;
    int res = 0;
    for (int i = 0; i < (int)h.size(); i++) {
        while (!st.empty() && h[st.top()] >= h[i]) {
            int height = h[st.top()];
            st.pop();
            int width = st.empty() ? i : i - st.top() - 1;
            res = max(res, height * width);
        }
        st.push(i);
    }
    return res;
}

```

7.2. FracBinarySearch

Tìm phân số $\frac{p}{q}$ nhỏ nhất trong đoạn $[0, 1]$ sao cho $f(\frac{p}{q})$ là đúng, với $p \leq m_p, q \leq m_q$

```

using ll = long long;
struct Frac {
    ll p, q;
};

template <class Func>
Frac FracLowerBound(Func f, ll maxq, ll maxp) {
    Frac lo{0, 1}, hi{1, 1}, mid; // Set hi to 1/0 for (0, inf)
    if (f(lo)) return lo;
    assert(f(hi));
    for (int it = 0, dir = 1; it < 3 || !dir; ++it) {
        // invariant: f(lo) == !dir, f(hi) == dir
        for (ll step = 1, adv = 1, now = 0;; adv ? step *= 2 : step /= 2) {
            now += step;
            mid = {lo.p * now + hi.p, lo.q * now + hi.q};
            if (abs(mid.p) > maxp || mid.q > maxq || f(mid) != dir) now -= step, adv = 0;
            if (!step) break;
        }
        if (mid.p != hi.p) it = 0;
        hi = lo;
        lo = mid;
        dir = !dir;
    }
    // (lo, hi) are consecutive with f(lo) == 0, f(hi) == 1
    return hi;
}

```

7.3. MaximalCliques

Chạy một hàm nào đó duyệt qua tất cả các clique của một đồ thị trong $O(3^{\frac{n}{3}})$.

```

// Usage: cliques(g, [&](const bs &clique)
{ callback }, ~bs(n), bs(n), bs(n));

```

```
template <class F>
void cliques(vector<bs>& eds, F f, bs P, bs X, bs R)
{
    f(R);
    if (!P.any() && !X.any()) return;
    // if only need to find all maximal cliques
    // auto q = (P | X).find_first();
    // auto cand = P & ~eds[q];
    for (int i = 0; i < sz(eds); ++i) {
        if (P[i]) {
            R[i] = 1;
            cliques(eds, f, P & eds[i], X & eds[i], R);
            R[i] = P[i] = 0, X[i] = 1;
        }
    }
}
```

7.4. MaximumClique

Tìm nhanh một clique lớn nhất. Dùng để giải Maximum Independent Set bằng cách tính maximum clique của phần bù.

```
struct Maxclique {
    ld limit = 0.025, pk = 0;
    struct Vertex {
        int i, d = 0;
    };
    typedef vector<Vertex> vv;
    vector<bs> e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old; // qmax = vertices in maximum
    clique, q = current clique
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) {
            for (auto j : r) v.d += e[v.i][j.i];
        }
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        for (int i = 0; i < sz(r); ++i) r[i].d = min(i, mxD) + 1;
    }
}
```

```

}
void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
        if (sz(q) + R.back().d <= sz(qmax)) return;
        q.push_back(R.back().i);
        vv T;
        for (auto v : R)
            if (e[R.back().i][v.i]) T.push_back({v.i});
        if (sz(T)) {
            if (S[lev]++ / ++pk < limit) init(T);
            int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
            C[1].clear(), C[2].clear();
            for (auto v : T) {
                int k = 1;
                auto f = [&](int i) { return e[v.i][i]; };
                while (any_of(all(C[k]), f)) k++;
                if (k > mxk) mxk = k, C[mxk + 1].clear();
                if (k < mnk) T[j++].i = v.i;
                C[k].push_back(v.i);
            }
            if (j > 0) T[j - 1].d = 0;
            for (int k = mnk; k <= mxk; ++k) {
                for (int i : C[k]) T[j].i = i, T[j++].d = k;
            }
            expand(T, lev + 1);
        } else if (sz(q) > sz(qmax))
            qmax = q;
        q.pop_back(), R.pop_back();
    }
}
vi maxClique() {
    init(V), expand(V);
    return qmax;
}
Maxclique(vector<bs> conn) : e(conn), C(sz(e) + 1),
S(sz(C)), old(S) {
    for (int i = 0; i < sz(e); ++i) V.push_back({i});
}
```

```
};
```

7.5. Frievalds

Kiểm tra xác suất tích ma trận $AB = C$ trong $O(Tn^2)$. Xác suất sai là 2^{-T} .

```
int Freivalds(Mat a, Mat b, Mat c) {
    int n = a.n, iteration = 40;
    Mat zero(n, 1), r(n, 1);
    while (iteration--) {
        for (int i = 0; i < n; i++) r.a[i][0] = rnd() % 2;
        Mat ans = (a * (b * r)) - (c * r);
        if (ans != zero) return 0;
    }
    return 1;
}
```

8. Hình

Các thuật toán hình có đa giác, nếu không chú thích gì, thì hoạt động với mọi loại đa giác (lồi, lõm, tự cắt). Khi không còn bài gì để làm nữa thì hăng làm hình.

8.1. Point

```
#pragma once

template <class T>
int sgn(T x) { return (x > 0) - (x < 0); }
template <class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
    bool operator==(P p) const { return tie(x, y) == tie(p.x, p.y); }
    P operator+(P p) const { return P(x + p.x, y + p.y); }
    P operator-(P p) const { return P(x - p.x, y - p.y); }
}
```

```

P operator*(T d) const { return P(x * d, y * d); }
P operator/(T d) const { return P(x / d, y / d); }
T dot(P p) const { return x * p.x + y * p.y; }
T cross(P p) const { return x * p.y - y * p.x; }
T cross(P a, P b) const { return (a -
*this).cross(b - *this); }
T dist2() const { return x * x + y * y; }
T dist() const { return sqrt(dist2()); }
// angle to x-axis in interval [-pi, pi]
T angle() const { return atan2l(y, x); }
P unit() const { return *this / dist(); } // makes
dist()=1
P perp() const { return P(-y, x); } //
rotates +90 degrees
P normal() const { return perp().unit(); }
// returns point rotated 'a' radians ccw around the
origin
P rotate(ld a) const {
    return P(x * cos(a) - y * sin(a), x * sin(a) + y
* cos(a));
}
friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")";
}
};

```

8.2. TrianglePointCount

```

#include "Point.h"
template <class P, int MAXN, int MAXM>
struct TrianglePointCount {
    // Bảng lưu trạng thái: side[i][j] = bitset các
    điểm B nằm bên trái vector A[i]->A[j]
    bitset<MAXM> side[MAXN][MAXN];
    const vector<P>& A; // Tham chiếu tới mảng A để
    kiểm tra hướng khi truy vấn
    // Constructor: Thực hiện Precomputation O(N^2 * M)
    TrianglePointCount(const vector<P>& A, const
vector<P>& B)
        : A(A) {
        int n = sz(A), m = sz(B);
        for (int i = 0; i < n; ++i) {

```

```

        for (int j = 0; j < n; ++j) {
            if (i == j) continue;
            P vecIJ = A[j] - A[i]; // Vector A[i] -> A[j]
            for (int k = 0; k < m; ++k) {
                P vecIK = B[k] - A[i]; // Vector A[i] ->
                B[k]
                // Nếu B[k] nằm thực sự bên trái A[i]->A[j]
                (cross product > 0)
                if (vecIJ.cross(vecIK) > 0) side[i][j][k] =
                1;
            }
        }
    }
    // Truy vấn: Đếm số điểm B nằm trong tam giác A[a],
    A[b], A[c]
    // Độ phức tạp: O(M/64) ~ O(1)
    int query(int a, int b, int c) {
        // Kiểm tra hướng của tam giác
        auto area = A[a].cross(A[b], A[c]);
        if (area == 0) return 0; // Tam giác suy biến
        (thẳng hàng)
        if (area > 0) {
            // Ngược chiều kim đồng hồ (CCW): A->B->C
            // Điểm trong tam giác phải nằm trái AB, trái
            BC, VÀ trái CA
            return (side[a][b] & side[b][c] & side[c]
[a]).count();
        } else {
            // Cùng chiều kim đồng hồ (CW): A->C->B là CCW
            return (side[a][c] & side[c][b] & side[b]
[a]).count();
        }
    }
};

```

8.3. SideOf

```

#include "Point.h"
template <class P>
int sideOf(P s, P e, P p) {
    return sgn(s.cross(e, p));
}

```

```

}

template <class P>
int sideOf(const P& s, const P& e, const P& p, ld
eps) {
    auto a = (e - s).cross(p - s);
    ld l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}

```

8.4. ClosestPair

```

#include "Point.h"
typedef Point<i64> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<i64, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (i64)sqrt(ret.first), 0};
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi =
S.upper_bound(p + d);
        for (; lo != hi; ++lo) ret = min(ret, {( *lo -
p).dist2(), { *lo, p } });
        S.insert(p);
    }
    return ret.second;
}

```

8.5. ConvexHull

Trả về bao lồi của tập điểm theo CCW. Nếu muốn tính cả điểm nằm trên biên, sửa $\leq$ thành $<$.

```

#include "Point.h"
template <class P>
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
}

```

```

sort(all(pts));
vector<P> h(2 * sz(pts) + 2);
int s = 0, t = 0;
for (int it = 2; it--; s = --t, reverse(all(pts)))
{
    for (P p : pts) {
        while (t >= s + 2 && h[t - 2].cross(h[t - 1],
            p) <= 0) t--;
        h[t++] = p;
    }
}
return {h.begin(), h.begin() + t - (t == 2 && h[0]
== h[1])};
}

```

8.6. OnSegment

```

#include "Point.h"
template <class P>
bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <=
0;
}

```

8.7. SegmentIntersection

```

#include "OnSegment.h"
#include "Point.h"
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b), oc =
a.cross(b, c),
    od = a.cross(b, d);
    // Checks if intersection is single non-endpoint
    point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
}

```

```

if (onSegment(a, b, d)) s.insert(d);
return {all(s)};
}

```

8.8. LineDistance

```

#include "Point.h"
template <class P>
ld lineDist(const P& a, const P& b, const P& p) {
    return (ld) (b - a).cross(p - a) / (b - a).dist();
}

```

8.9. LineIntersection

```

#include "Point.h"
#include "Line.h"
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
tuple<T4, T4, T2> LineIntersection(Line m, Line n) {
    T2 d = (T2)m.a * n.b - (T2)m.b * n.a; // assert(d);
    T4 x = (T4)m.c * n.b - (T4)m.b * n.c;
    T4 y = (T4)m.a * n.c - (T4)m.c * n.a;
    return {x, y, d}; // (x/d, y/d) is intersection.
}

```

8.10. LineHullIntersection

```

#include "Point.h"
#define cmp(i, j) sgn(dir.perp().cross(poly[(i) % n]
- poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 +
n) < 0

```

```

template <class P>
int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi :
lo) = m;
    }
    return lo;
}
#define cmpl(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpl(endA) < 0 || cmpl(endB) > 0) return {-1,
-1};
    array<int, 2> res;
    for(int i = 0; i < 2; ++i) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) %
n;
            (cmpl(m) == cmpl(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpl(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpl(res[0]) && !cmpl(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) %
sz(poly)) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    return res;
}

```

8.11. LineProjectionReflection

Trả về chân đường vuông góc/điểm đối xứng (tùy vào refl=false/true) của điểm p qua đường ab. Các điểm phải là số thực, cần thận tràn số.

```
#include "Point.h"

template <class P>
P lineProj(P a, P b, P p, bool refl = false) {
    P v = b - a;
    return p - v.perp() * (1 + refl) * v.cross(p - a) /
        v.dist2();
}
```

8.12. CircleLine

Định nghĩa của đường thẳng dạng $ax + by = c$ với $a, b, c \in \mathbb{Z}$

```
#include "Point.h"

template <class P>
vector<P> circleLine(P c, ld r, P a, P b) {
    P ab = b - a;
    ld s = a.cross(b, c);

    // calculate intersection, return vector<P>
    P p = a + ab * (c - a).dot(ab) / ab.dist2();
    ld h2 = r * r - s * s / ab.dist2();
    if (h2 < 0) return {};
    if (h2 == 0) return {p};
    P h = ab.unit() * sqrt(h2);
    return {p - h, p + h};

    // calculate smaller part area, return ld
    // ld dist = fabs(s) / sqrt(ab.dist2());
    // assert(dist <= r);
    // ld theta = 2.0 * acos(dist / r);
    // ld area = 0.5 * r * r * (theta - sin(theta));
    // return area;
}
```

8.13. CircleIntersection

```
#include "Point.h"

typedef Point<ld> P;
bool circleInter(P a, P b, ld r1, ld r2, pair<P, P>*
out) {
    if (a == b) {
        assert(r1 != r2);
        return false;
    }
    P vec = b - a;
    ld d2 = vec.dist2(), sum = r1 + r2, dif = r1 - r2,
    p = (d2 + r1 * r1 - r2 * r2) / (d2 * 2), h2 = r1
    * r1 - p * p * d2;
    if (sum * sum < d2 || dif * dif > d2) return false;
    P mid = a + vec * p, per = vec.perp() *
    sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

8.14. CircleTangents

Tìm các tiếp tuyến ngoài của hai hình tròn, hoặc các tiếp tuyến trong nếu r2 âm.

- Có thể trả về 0, 1 hoặc 2 tiếp tuyến:
- 0 nếu một hình tròn chứa (hoặc chồng lên nhau, trong trường hợp nội tiếp, hoặc nếu hai hình tròn giống hệt nhau) hình tròn kia.
- 1 nếu hai hình tròn tiếp xúc với nhau (trong trường hợp này first = second và đường tiếp tuyến vuông góc với đường nối giữa tâm).
- first và second tương ứng cho biết các điểm tiếp xúc tại hình tròn 1 và hình tròn 2.
- Để tìm các tiếp tuyến của một hình tròn với một điểm, hãy đặt r2 = 0.

```
#include "Point.h"

template <class P>
vector<pair<P, P>> tangents(P c1, ld r1, P c2, ld r2) {
    P d = c2 - c1;
    ld dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
```

```
if (d2 == 0 || h2 < 0) return {};
vector<pair<P, P>> out;
for (ld sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
}
if (h2 == 0) out.pop_back();
return out;
}
```

8.15. Circumcircle

```
#include "Point.h"

typedef Point<ld> P;
ld ccRadius(const P& A, const P& B, const P& C) {
    return (B - A).dist() * (C - B).dist() * (A -
    C).dist() /
        abs((B - A).cross(C - A)) / 2;
}
P ccCenter(const P& A, const P& B, const P& C) {
    P b = C - A, c = B - A;
    return A + (b * c.dist2() - c * b.dist2()).perp() /
        b.cross(c) / 2;
}
```

8.16. MinimumEnclosingCircle

```
#include "Circumcircle.h"

pair<P, ld> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    ld r = 0, EPS = 1 + 1e-12;
    for (int i = 0; i < sz(ps); ++i) {
        if ((o - ps[i]).dist() > r * EPS) {
            o = ps[i], r = 0;
            for (int j = 0; j < i; ++j) {
                if ((o - ps[j]).dist() > r * EPS) {
                    o = (ps[i] + ps[j]) / 2;
                    r = (o - ps[i]).dist();
                    for (int k = 0; k < j; ++k) {
```

```

        if ((o - ps[k]).dist() > r * EPS) {
            o = ccCenter(ps[i], ps[j], ps[k]);
            r = (o - ps[i]).dist();
        }
    }
}
}
return {o, r};
}

```

8.17. CirclePolygonIntersection

Trả về diện tích phần giao của đường tròn với đa giác trong $O(n)$

```

#include "Point.h"

typedef Point<ld> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
ld circlePoly(P c, ld r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p) / d.dist2(), b = (p.dist2() - r
            * r) / d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)), t = min(1., -a
            + sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + u.cross(v) / 2 + arg(v,
            q) * r2;
    };
    auto sum = 0.0;
    for (int i = 0; i < n; ++i) i, 0, sz(ps)) sum +=
        tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}

```

8.18. InsidePolygon

```

#include "OnSegment.h"
#include "Point.h"

template <class P>
bool inPolygon(vector<P>& p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    for (int i = 0; i < n; ++i) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        // or: if (segDist(p[i], q, a) <= eps) return !
        strict;
        cnt ^= ((a.y < p[i].y) - (a.y < q.y)) *
            a.cross(p[i], q) > 0;
    }
    return cnt;
}

```

8.19. PolygonCenter

```

#include "Point.h"

typedef Point<ld> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0);
    ld A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}

```

8.20. PolygonArea

Trả về 2 lần diện tích có dấu của đa giác.

```

#include "Point.h"

template <class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);

```

```

    for (int i = 0; i < n; ++i) i, 0, sz(v) - 1) a +=
        v[i].cross(v[i + 1]);
    return a;
}

```

8.21. PointInsideHull

```

#include "OnSegment.h"
#include "Point.h"
#include "SideOf.h"

typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict =
    true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0],
        l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0],
        l[b], p) <= -r) return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}

```

8.22. HullDiameter

```

#include "Point.h"

// S must already be a convex hull
template<class P>
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<i64, array<P, 2>> res({0, {S[0], S[0]}});
    for (int i = 0; i < j; ++i) {
        for (; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i],
                S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] -
                S[i]) >= 0) break;

```

```

    }
}
return res.second;
}

```

8.23. Minkowski

Tính tổng của 2 bao lồi trong $O(n + m)$.

```

#include "Point.h"

vector<Point> MinkowskiSum(vector<Point> P,
vector<Point> Q) {
    int n = P.size(), m = Q.size();
    vector<Point> R = {P[0] + Q[0]};
    for (int i = 1, j = 1; i < n || j < m; ) {
        if (i < n && (j == m || cross(P[i] - P[i - 1],
            Q[j] - Q[j - 1]) > 0)) {
            R.push_back(R.back() + P[i] - P[i - 1]);
            ++i;
        } else {
            R.push_back(R.back() + Q[j] - Q[j - 1]);
            ++j;
        }
    }
    return R;
}

```

)